When to Use a Service Mesh in Cloud-Native Architectures

30 October 2024 - ID G00818217 - 29 min read

By: Steve Deng, Simon Richard

Service mesh technologies address cross-cutting service connectivity concerns in microservices architectures. Application technical professionals must understand the benefits and complexities of service mesh patterns and apply them to new and existing highly dynamic cloud-native architecture.

Overview

Key Findings

- Service mesh technologies manage communications between services (that is, east-west traffic), often in a way that is transparent to the service implementation. However, some service meshes include ingress and egress gateways that can be configured to enforce API management policies (for north-south traffic) at the entry and exit points of a service mesh.
- Modern service meshes commonly use sidecar proxies to intercept communication between services and implement cross-cutting concerns. Other approaches include embedded proxy, node proxy, split-layer proxy and specialized network-centric proxy.
- Cloud providers are offering service mesh as a stand-alone managed service or as a packaged component of a container management system. This lowers the bar of entry to service mesh adoption by reducing operational burden and increasing availability and support.
- There are viable alternatives to service mesh. They offer more targeted features or more clearly delineated separation of concern for platform engineering, networking and DevOps teams. Options include Kubernetes container network interface (CNI) plugins, containerized API gateways and microservices runtime frameworks (such as Dapr).

Recommendations

As an application technical professional responsible for distributed application architecture, you should:

This research note is restricted to the personal use of .

- Use service mesh technology to enforce best practices for life cycle management, security, policy management and enforcement, and monitoring in dynamic and large-scale microservices implementations.
- Avoid jumping into a service mesh implementation if you have not adopted continuous integration/continuous delivery (CI/CD) automation for your multigrain services or if you have yet to establish deployment and operational models for Kubernetes.
- Use a combination of API gateway, ingress gateway and service mesh to support mediation of API and service communication both within and across service and business domains.
- Avoid service mesh if you are still evaluating or experimenting with a container orchestration platform such as Kubernetes for hosting containerized services. Establish operational readiness and maturity with Kubernetes first before considering service mesh and its alternatives.

Analysis

Service mesh is a distributed computing pattern, implemented as middleware, that optimizes and regulates communications between application services. It provides proxy and lightweight mediation for service-to-service communications and supports functions such as authentication, authorization, encryption, service discovery, request routing and communication traceability.

This research helps application technical professionals answer the question: "What is a service mesh and when should I use one?"

What Is a Service Mesh?

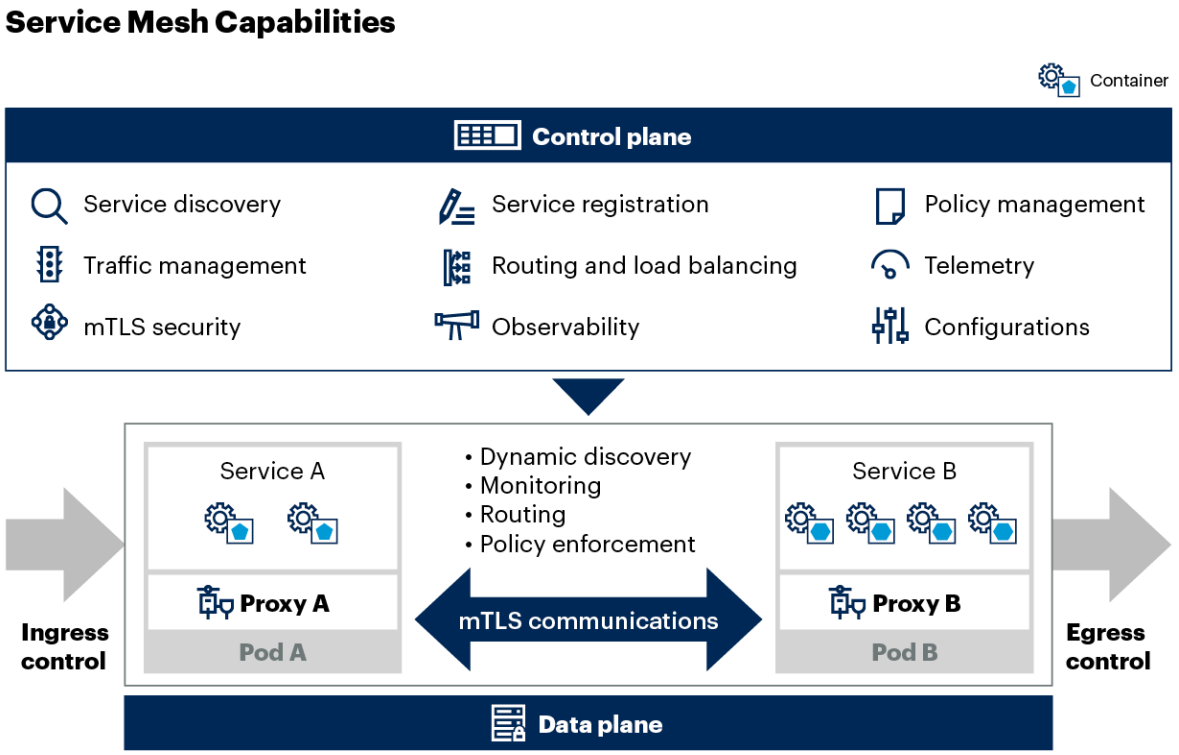
A service mesh is a component of a cloud-native software platform that implements distributed computing patterns and provides tools to deal with cross-cutting concerns — such as service discovery, traffic routing, observability and encryption — among various services. This pattern is analogous to an "aspect-oriented programming" approach to software development, in which a frequently used piece of functionality is implemented independent of the program logic and instantiated at runtime.

This research note is restricted to the personal use of .

Figure 1 illustrates the control plane and data plane of a service mesh and their capabilities with ingress and egress gateways. Service mesh implementations consist of a control plane and a data plane, which together provide key capabilities to keep communications between services loosely coupled, reliable, secure and flexible. The control plane offers an interface for operators to enter and manage policies and communicate with the data plane. The data plane manages the communications between services by applying and enforcing the policies set by the control plane.

Download All Graphics in This Material

Figure 1: Service Mesh Capabilities



Source: Gartner
818217_C

Gartner

Control Plane

The control plane provides management capabilities including, but not limited to:

- **Service registry and discovery:** Maintain a service registry that allows a service instance to register itself when it enters a system and deregister as it leaves a system. A calling service, with the help of a sidecar, can query the service registry to get the location of a service provider and then communicate with the provider.

This research note is restricted to the personal use of .

- **Traffic management policies:** Manage traffic policies that define how requests are routed to an instance of the target service. These policies are (typically) delivered to the sidecars via a data plane configuration — for example, xDS in the case of Envoy Proxy. Traffic from a client application may be prioritized, or traffic may be selectively routed to a different version of a service in support of a canary-release strategy, AB testing or a circuit breaker. Traffic management policies may include traffic routing, load balancing, throttling, rate limiting, quota, retries back-off, circuit breaking, blue/green and canary deployment.

- **Observability and telemetry policies:** Traffic between services can be instrumented for logging and debugging purposes by the service mesh with minimal to no modifications to the service code. These policies can be enforced by the data plane to gather and report back the desired metrics and state of health to the control plane. Architectural observability is an added benefit because you can view the actual calls between services.

- **Service identity:** Manage and maintain a mapping of service identities to the services they can access. Also maintain a log of which service accessed which other service at what time. Service identity of the calling and provider service is validated by Mutual Transport Layer Security (mTLS). Moreover, authorization based on the end-user identity can be achieved through credential propagation and validation of JSON Web Tokens (JWT).

- **Secure service-to-service communication:** Secure communications between services can be transparently encrypted without user intervention. The control plane provides functionality, such as certificate generation from a certificate authority (CA) and certificate rotation, and pushes these certificates and related configuration data to the data plane.

- **Configuration:** Facilitate policies and service configuration by providing a management interface and a configuration store.

Data Plane

The data plane enforces the policies set by the control plane that are applied to the services running in the control plane. The data plane provides capabilities including, but not limited to: [1]

- **Service registration and discovery:** Register or deregister a service instance upon launch or shutdown to the service registry. The data plane component also queries the control plane to resolve what other services are available.

This research note is restricted to the personal use of .

- **Health checking:** Validate that service instances respond to basic health checks and report their state of health to the control plane. The data plane also confirms that services returned from service discovery are healthy and ready to accept requests.
- **Traffic routing:** When an inbound request for a specific resource is made, such as via a REST request to a specific endpoint in the current service instance, the data plane can route the request to the appropriate service. The data plane automatically copes with the rapidly shifting topology of an autoscaling containerized platform.
- **Load balancing:** Once the requested service endpoint has been set from the traffic routing policy, follow the relevant load-balancing policy to send the request to the correct service instance.
- **Authentication and authorization:** Follow the identity and encryption policies set by the control plane. Confirm that the service caller has the correct permissions to invoke the requested endpoint (for authorization) and is correctly identified via a mechanism such as mTLS (for authentication). The service mesh will manage the issuance of X.509 certificates for services.
- **Observability:** Collect detailed performance and usage statistics, logs and other pertinent data, such as distributed tracing data for each request. Deliver this data to a telemetry provider. The control plane also provides observability functionality. It uses telemetry data and can route data for further analytical processing.

Envoy, an open-source project that originated at Lyft, has become the most prevalent data plane. Envoy is the default data plane for Istio and the recommended data plane for HashiCorp Consul, Kong Kuma and Amazon Web Services (AWS) App Mesh. Other data planes include Istio Ambient Mesh and Linkerd proxy from Buoyant.

Components of a Data Plane

The data plane sits on the path of a service interaction and is responsible for policy enforcement along that path. The components of a data plane typically include a collection of policy enforcement points, in the form of proxies, that takes care of service-to-service communications — or east-west traffic. Optional ingress and egress gateways manage inbound and outbound (north-south) traffic going in and out of the mesh.

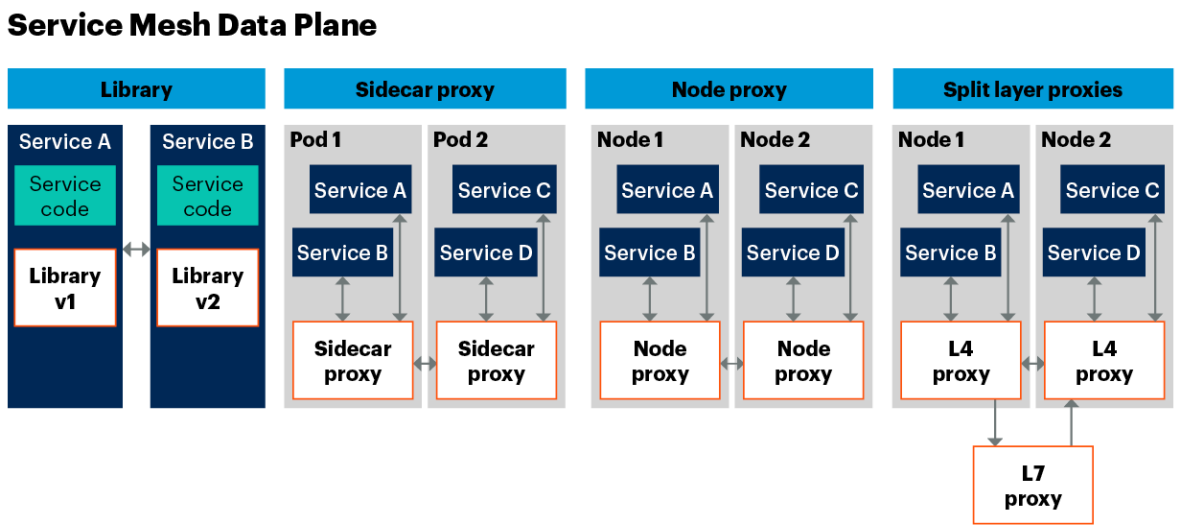
For east-west traffic, the data plane (see Figure 2) typically takes one of the following forms:

This research note is restricted to the personal use of .

- A library or proxyless component that is embedded in a stand-alone application or service (for example, proxyless gRPC, which implements Envoy's xDS APIs).
- A sidecar proxy, deployed adjacent to the service instances (for example, within a Kubernetes pod) that it manages. Examples include Istio proxy and Linkerd proxy.
- A node proxy, deployed to a node of a cluster or a host, that is responsible for the services within that node or host. Traefik proxy is an example of a node proxy.
- A split-layer proxy that is composed of two separate components — one base proxy that operates in OSI L4 and one L7 (application) proxy:
  - The L4 base proxy, typically deployed in the node, is responsible for mutual TLS channel encryption and Transmission Control Protocol (TCP) routing and observability to minimize network latency.
  - The L7 proxy takes on application-level security and authorization policy, provides protocol-level observability and enforces rich traffic management such as canary releases on an as-needed basis.

For example, Istio Ambient Mesh bundles zTunnel (L4) and Waypoint proxy (L7) for its data plane; Cilium Mesh uses the eBPF program in the kernel for its L4 proxy and Envoy for its L7 proxy.

Figure 2: Service Mesh Data Plane



Source: Gartner
818217_C

Gartner

This research note is restricted to the personal use of .

For north-south traffic, an ingress gateway manages inbound traffic that enters the mesh or cluster and hands off to service mesh policy enforcement points (PEPs) to manage east-west traffic within the mesh or cluster. Similarly, an egress gateway controls the outbound traffic from within the mesh or cluster, and routes them to external services or applications. Not all service mesh implementations include ingress or egress gateways as part of their offerings. For example, Istio provides an ingress/egress gateway as part of its offering, while Linkerd (as of v2.11) doesn't provide a built-in ingress gateway "for reasons of simplicity and composability." [2]

Ingress and egress gateways are the nexus where API gateways and service meshes intersect. When integrated with API management, ingress and egress gateways can enable end-to-end visibility and control of API, and service traffic across hybrid and multicloud.

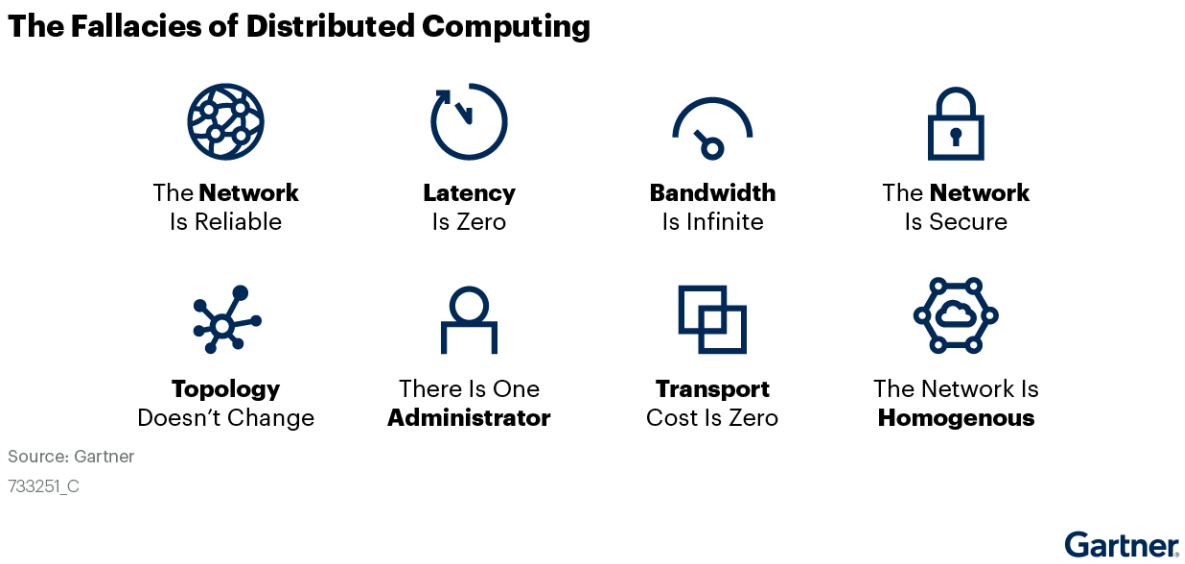
Service Meshes Solve Many Challenges of Distributed Computing

Service mesh focuses on solving many of the problems that are shifted to the outer architecture by applying consistent patterns and best practices. This still requires developers to understand the inherent trade-offs that the architecture brings. These are best summed up as the fallacies of distributed computing, [3] as illustrated in Figure 3.

Service mesh patterns attempt to systematically address the fallacies of distributed computing in a consistent and repeatable manner.

This research note is restricted to the personal use of .

Figure 3: The Fallacies of Distributed Computing



A full discussion of the fallacies of distributed computing is out of scope for this document, but application technical professionals must pay particular attention to network complexity, latency and reliability. Microservices architecture (MSA) introduces additional concerns around these three fallacies, when compared to a monolithic architecture.

The additional complexity that networking brings needs to be managed. But as companies such as Lyft have shown with the Envoy Proxy, the benefits in developer productivity make this complexity worthwhile (see Table 1).

This research note is restricted to the personal use of .

Table 1: Service Mesh and the Fallacies of Distributed Computing

(Enlarged table in Appendix)

Fallacies	Service mesh functionality
■ The network is reliable. ■ Latency is zero. ■ Bandwidth is infinite. ■ Transport cost is zero.	■ Configurable resiliency patterns to deal with failures using retries, load balancing and circuit breakers. ■ Additional error-handling patterns to deal with network outages and variations in network reliability and capacity.
■ The network is secure.	■ Systematically enforced security policies across all services within the service mesh.
■ Topology doesn't change.	■ Dynamically adjusts the network to changing topology and application requirements.
■ There is one administrator.	■ API-driven access to the control plane to allow developers and platform owners and operators to jointly manage the entire structure of the service mesh and dynamically reconfigure as needed.
■ The network is homogeneous.	■ Abstraction of the application from Open Systems Interconnection model Layer 3/Layer 4 (L3/L4) and any specific implementation details.

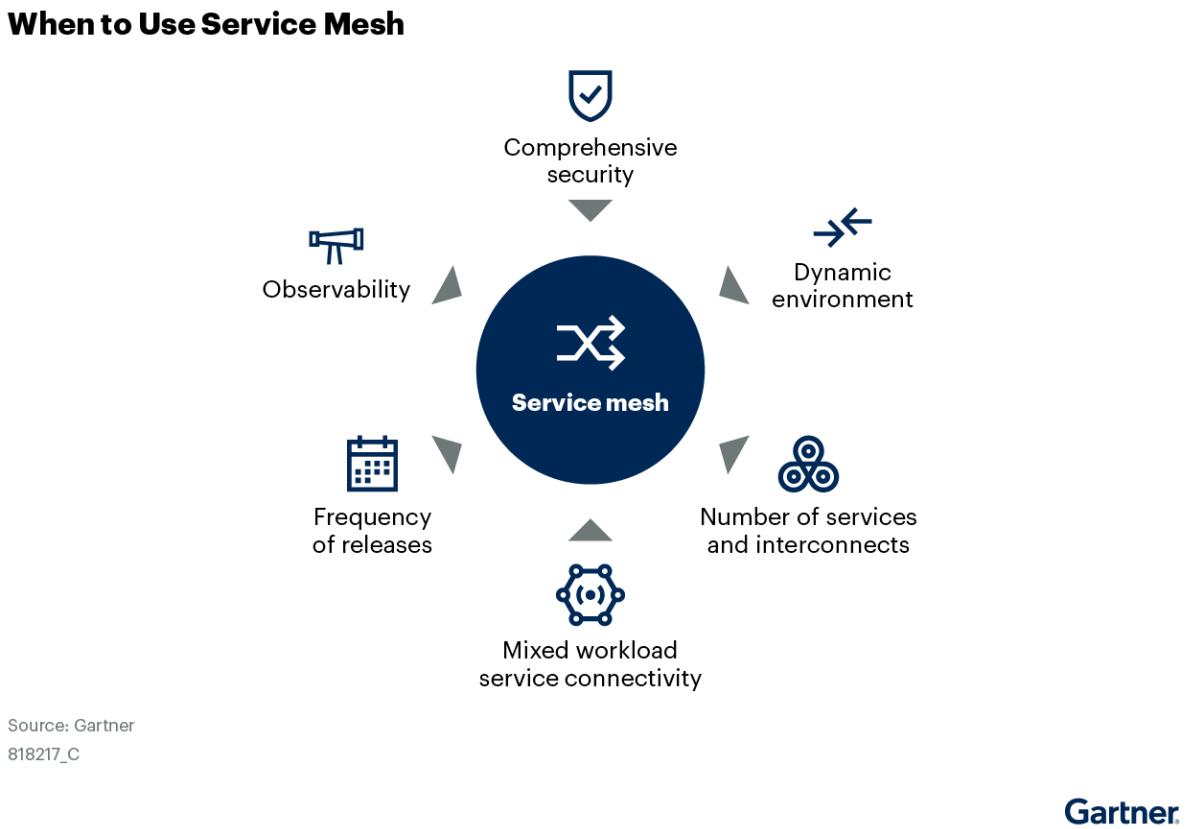
Source: Gartner

When to Use a Service Mesh

As a rule of thumb, if you have a dynamic, frequently changing microservices environment with a high volume of east-west communications, you need a service mesh. Use the six criteria shown in Figure 4 to guide your decision when considering a service mesh.

This research note is restricted to the personal use of .

Figure 4: When to Use a Service Mesh



If you answer *yes* to any questions in at least **two** criteria below, you can benefit from a service mesh:

- **Comprehensive security:** Do you want to secure and protect your interservice communication? Do you need consistent access policy enforcement — for example, using mTLS for service authentication and channel encryption, and access tokens for service authorization? Do you need the ability to automate the issuance of mTLS certificates to your services at scale using your public and internal CA? Regardless, service mesh can be foundational in the implementation of a security zero-trust architecture.

- **Dynamic versus static environment:** Do you have an environment with highly dynamic topology, with frequent changes in the underlying networking details — that is, multiple services scaling up and down, and highly dynamic workloads? Are you deploying services across hybrid clouds or actively migrating services from on-premises data centers to the cloud? Alternatively, if your environment is relatively stable, such as with a monolithic application that has been repackaged into containers for ease of management purposes, a service mesh will be less useful.

This research note is restricted to the personal use of .

- **Number of services and service interconnects:** Have you reached a number of services — generally 10 or more — in your architecture? Does each service have, or scale to, five or more instances at any point? Are your services generating a significant amount of east-west network traffic? A fair amount of east-west traffic and potentially dynamic service-to-service interconnect topology are good indicators for service mesh adoption.

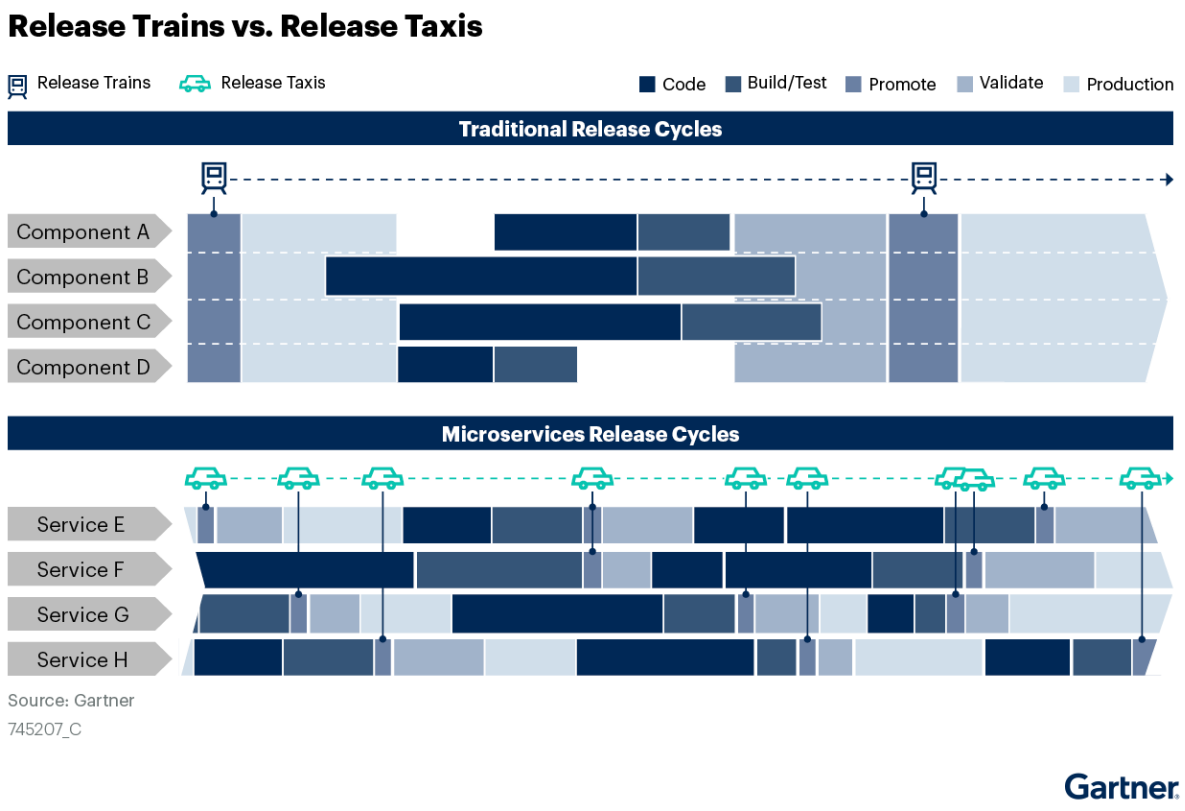
- **Multicluster or mixed workload service connectivity:** Does your service-to-service connectivity span multiple clusters, or containerized and virtual machine (VM) workloads? Do you have instances of a service deployed in an on-premises data center and the cloud, where you want to optimize the transaction flow by calling the closest instance? Some service meshes can facilitate cross-cluster service connectivity using a feature called service mesh federation, which allows multiple independently managed service meshes to propagate certain trusted service traffic.

- **Frequency of releases:** Are new versions of the services released on a weekly, daily or even more frequent basis? Are you using a taxi model rather than a train model for your releases, as indicated in Figure 5? Do you need features such as canary releases, blue/green deploys and traffic shaping for testing and to bring new services into production? Are you shifting quality to the right and left of your development life cycle? If you are doing frequent zero-downtime deployments of individual services and need to do canary releases and gradual redirection to new service versions, then a service mesh can help you articulate and execute such complex service routing policies. If you're doing monolithic "train" releases, your regression/integration tests are probably for the whole distributed monolith, and a service mesh is probably not needed in that case.

- **Observability:** Do you need to observe the interactions between services, such as tracking every transaction through the entire system, up to and including through specific code paths? Do you need the ability to understand service dependency through the system? Do you need to gather data about service usage in a consistent manner?

This research note is restricted to the personal use of .

Figure 5: Release Trains Versus Release Taxis



How Does a Service Mesh Fit Into Cloud-Native Architecture?

Applications that rely on the composition of multiple services can benefit from service mesh. Application architects look to cloud-native architecture and MSA to help increase developer velocity by decoupling application concerns — allowing developers to independently develop and release loosely coupled services. However, this decoupling comes at a cost. MSA brings with it the inherent complexity of managing distributed systems.

As discussed in Solution Path for Increasing Application Agility Using Microservices Principles, microservices architectures reorganize complexity into two areas:

- **Inner architecture:** The software architecture of an individual microservice and what it exposes to the outside world. This is defined by DevOps teams building and running services.

This research note is restricted to the personal use of .

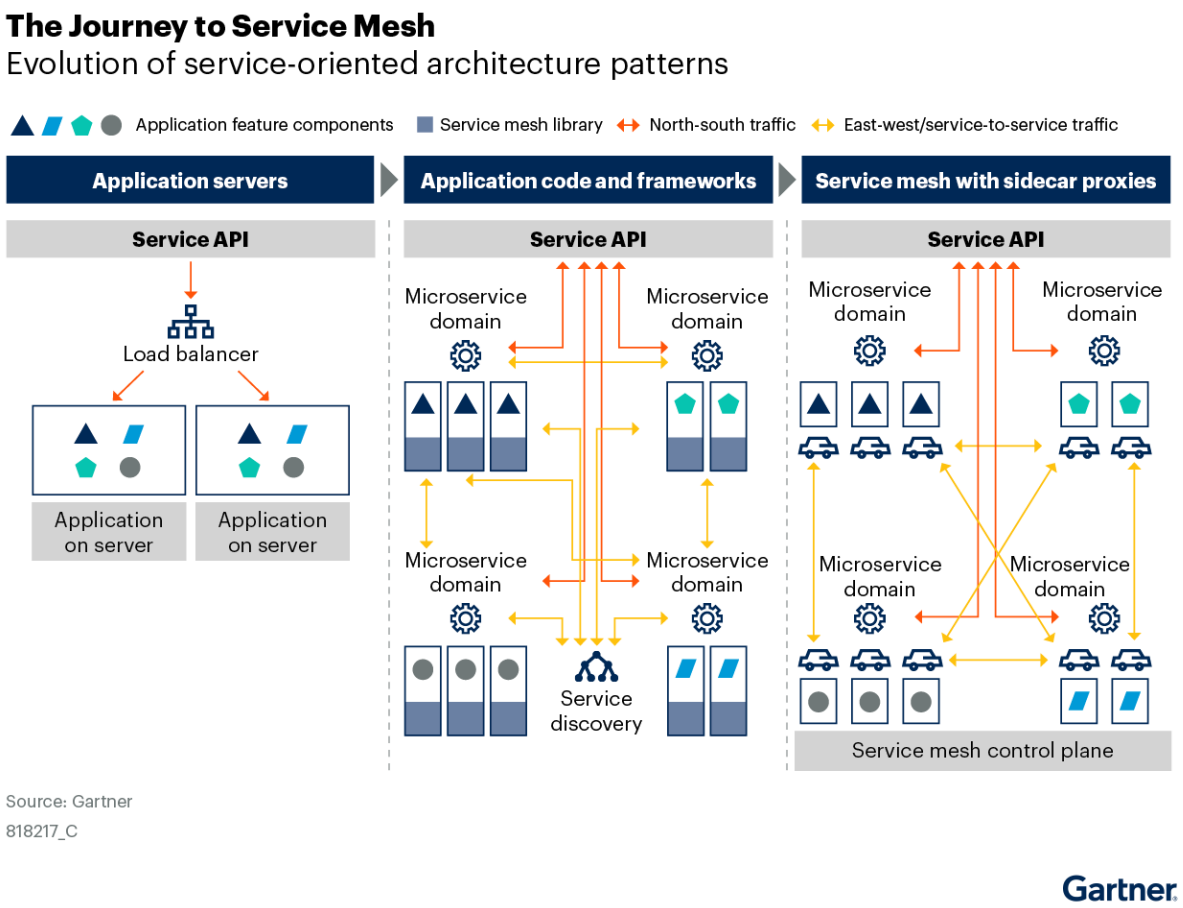
- **Outer architecture:** The architecture of the microservices infrastructure that provides an operating environment and distributed management platform on which your microservices will be built, deployed and executed. This is delivered and operated by platform engineering to support the work of DevOps teams. Service mesh is part of the outer architecture.

Service Meshes Simplify Many Best Practices for Cloud-Native Architecture

A set of best practices for MSAs has evolved over the last several years. Many of these best practices, such as observability, securing interservice communications, and certificate- and token-based authentication, were well-understood long before MSAs emerged — albeit they were inconsistently implemented with diverse technical solutions and differing development approaches. Previous approaches included first-generation service-oriented architectures (SOA), where much of this complexity was embedded in an application server, and framework-based approaches, such as those implemented in the Finagle framework from Twitter. Figure 6 illustrates the evolution of SOA patterns from application server, to application framework and service mesh with sidecar proxies. These approaches required development teams to know the implementation details of each one.

This research note is restricted to the personal use of .

Figure 6: The Journey to Service Mesh



Using a service mesh product or framework makes it far easier to consistently implement these patterns independent of development efforts. Essentially, service mesh emerged to make it easier to set up and manage best practices in a consistent manner. They have since evolved to a point where they make it possible to enforce policy consistently within a container orchestration platform, such as a Kubernetes cluster, or across multiple Kubernetes clusters and VM environments.

Where Do You Source the Service Mesh?

In recent years, open-source service mesh technologies have garnered vendor support and, as a result, are driving the adoption of service mesh. Organizations have several options when it comes to sourcing service mesh technologies:

- **Open-source service mesh:** A number of open-source service meshes are available today, with Istio and Linkerd being the most popular in terms of adoption. Other notable open-source service mesh projects include Kuma and Traefik Mesh.

This research note is restricted to the personal use of .

- **Cloud-provider-managed service mesh:** All major cloud providers offer some form of a managed service mesh as part of their cloud offerings. Google Cloud Platform has Anthos Service Mesh and Traffic Director. Microsoft Azure offers the Istio service mesh add-on for Azure Kubernetes Service (AKS), and AWS App Mesh focuses on providing "application-level networking to all your services." Alibaba Cloud Service Mesh (ASM) is a fully managed, Istio-compatible service mesh available on Alibaba Cloud. Tencent Cloud also offers a managed service mesh called Tencent Cloud Mesh (TCM).

- **Vendor commercially supported service mesh software:** Many vendors provide commercially supported service mesh software or services. Examples include Cisco Service Mesh Manager, Citrix Service Mesh, F5 (Aspen Mesh, NGINX Service Mesh, Volterra), HashiCorp Consul, Solo.io (Gloo Mesh Enterprise), Linkerd, Red Hat OpenShift Service Mesh, VMware Tanzu Service Mesh, Greymatter Application Networking Platform, and Tetrate Service Bridge.

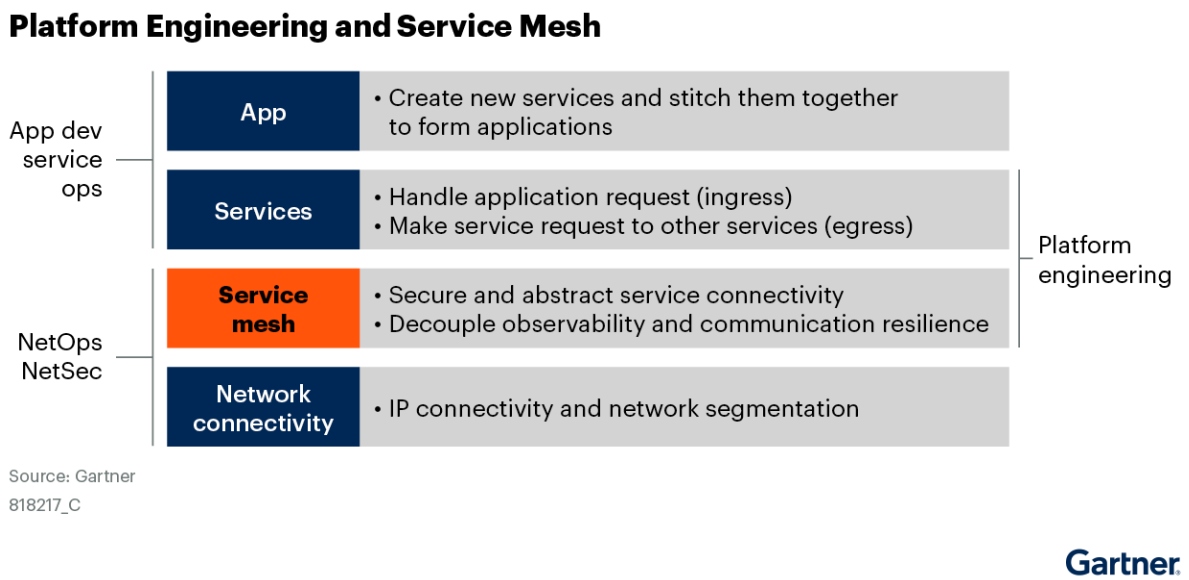
Who Should Manage the Service Mesh?

Service mesh patterns shift a variety of complex tasks away from the product development team and make them part of the outer architecture components of a cloud-native architecture.

Platform engineering is the team that delivers, maintains and improves a platform as a service (PaaS) for multiple agile application teams building custom software. The platform engineering team is concerned with the ongoing operations of the service mesh as part of the outer architecture of the platform (see Figure 7).

This research note is restricted to the personal use of .

Figure 7: Platform Engineering and Service Mesh



Developers will need the ability to interact with both the data and control planes of the service mesh via APIs and service-level policies, but they do not need to provision the service mesh or set the global policies that the service mesh implements. The service mesh can be configured to start with a set of well-balanced and level-headed defaults by the platform engineering team (see the examples in Table 2). The specific requirements for a particular service for tasks such as traffic routing or circuit breakers can be set by the product team or DevOps team responsible for that service.

This research note is restricted to the personal use of .

Table 2: Example Defaults for Service Mesh

(Enlarged table in Appendix)

Setting	Example default set by platform engineering	Customized settings by product teams
Service-to-service authentication	Set mTLS by default for all services to force services to encrypt their traffic by default.	Assign service or app identity. Set tags or labels as part of the CI/CD pipeline to provide metadata and context for policy selection and enforcement.
Ingress/egress rules	Set specific ports that traffic can pass through. For example, only traffic on port 80 or port 443 will be allowed.	Determine which services are exposed via the ingress/egress gateways.
Routing	Root-level Uniform Resource Identifier (URI) prefix delegation to namespaces owned by individual development teams.	Control the routing under the preassigned root URI prefix.
Monitoring	Export default monitoring statistics such as number of calls to a service.	Customize monitoring configuration and access metrics for services they own and operate.
Circuit breaking	Set a circuit breaker to trip when the maximum number of pending connections goes over a specific threshold.	Fine-tune the threshold for their services.

Source: Gartner

A useful construct to consider here is that of "mechanical sympathy," a phrase popularized by Martin Thompson and originally coined by Formula 1 champion Jackie Stewart. [4] Stewart believed that the best racing drivers had to have just enough understanding of how a motor racing car worked to get the best out of it. In the same vein, developers need to understand what a piece of functionality like service discovery or mTLS is for, and how to use it, but not necessarily the details of how to implement the functionality.

The platform engineering team has responsibility for maintaining and managing the service mesh control plane, creating sensible default policies and ensuring that the platform provisions the required components in the data plane. This can be viewed in terms of day-one and day-two operations. Day-one operations include activities such as the sourcing, selection and provisioning of the platform components (such as container registries, telemetry stores and container orchestration) and integration with existing systems, such as Lightweight Directory Access Protocol (LDAP) or Active Directory. Day-two operations include the ongoing maintenance of the platform, such as upgrading and updating platform software and adding capacity.

This research note is restricted to the personal use of .

To learn more about platform engineering, see Adopt Platform Engineering to Improve the Developer Experience.

Should I Use an Enterprise API Gateway, a Lightweight API Gateway, an Ingress Gateway or a Service Mesh?

API gateways manage the interactions between API consumers and published APIs (a published API is also referred to as an "outer API"). These interactions are frequently called north-south traffic. See MASA: Create Agile Application Architecture With Composable Apps, APIs and Services and Solution Path for Increasing Application Agility Using Microservices Principles for more details. Lightweight gateway can be used in a distributed manner to control API endpoints, frequently facilitating east-west traffic.

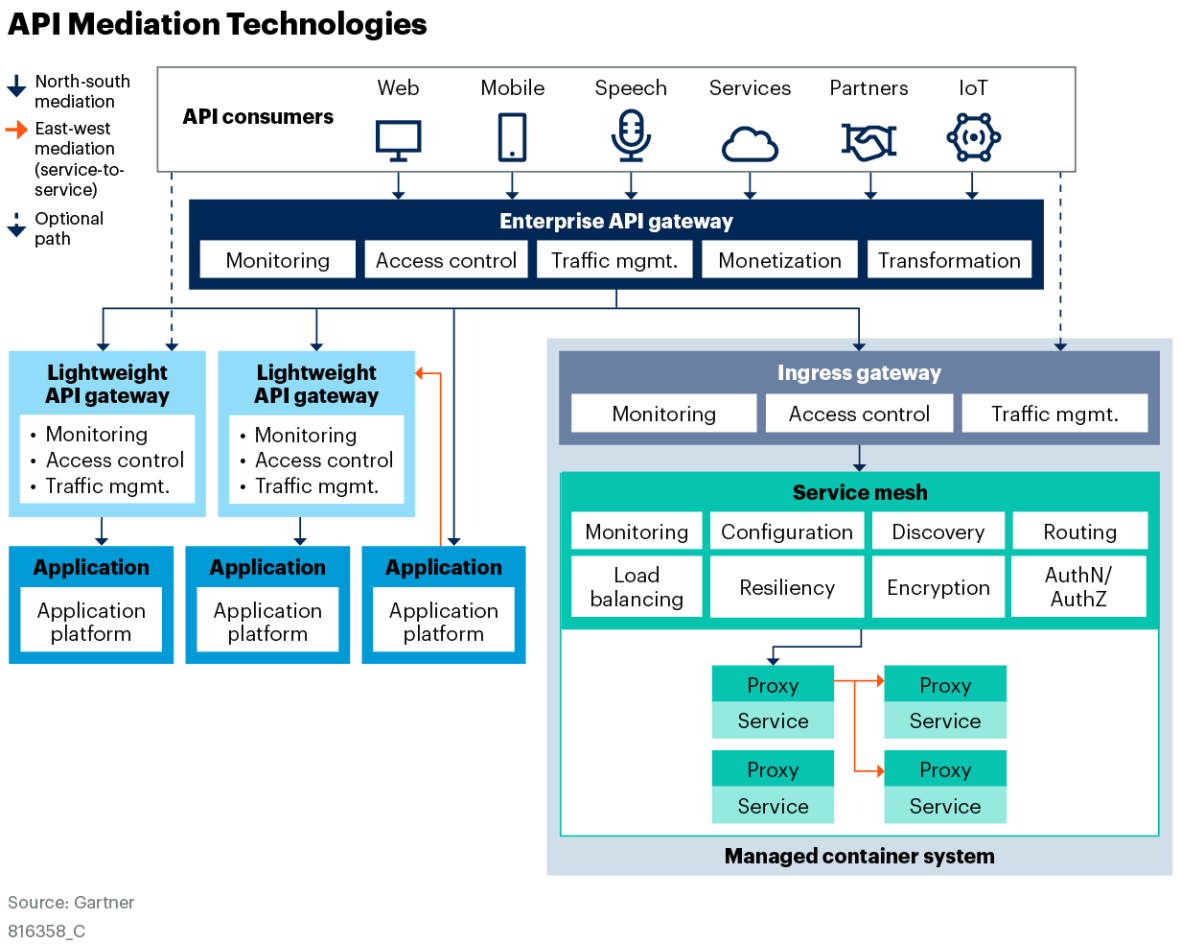
Organizations often confuse the roles of these four mediation technologies — enterprise API gateway, lightweight API gateway, ingress gateway and service mesh:

- An enterprise API gateway is typically located on the network perimeter to secure inbound and outbound API traffic. It is best-suited for external-facing, productized and published APIs with monetization potential. These gateways are often centrally managed by the organization and may be part of a broader APIM solution that includes API life cycle management and an API developer portal.
- A lightweight API gateway, aka microgateway, is designed to be distributed and deployed close to individual services. Unlike enterprise API gateways, which are often centrally managed as shared middleware, lightweight gateways can be individually deployed, configured and managed by a development team to meet the needs of the application.
- An ingress gateway refers to the entry point of a Kubernetes cluster or service mesh that bridges inbound (north-south) traffic from outside of the cluster to services within the cluster. An ingress gateway can be provisioned in a Kubernetes cluster as ingress controller or gateway API (without a service mesh), or as ingress for a service mesh such as Istio. Kubernetes gateway API expands the monolithic management model of ingress controller by creating delegated role-based separation of duties among the platform operator, NetOps, and application development teams.
- A service mesh focuses on the management of east-west traffic between services — the service-to-service interactions. A service mesh is not a replacement for an API gateway; however, many service meshes (such as Istio, Solo.io [Gloo Mesh Enterprise] and Kong Mesh) offer an ingress gateway that shares the capabilities of an API gateway.

This research note is restricted to the personal use of .

Manage your consumer-to-API interactions using enterprise API gateways or lightweight API gateways; manage your containerized service-to-service interactions using an ingress gateway or a service mesh, as illustrated in Figure 8 and described in Table 3.

Figure 8: Service Mesh and API Mediation



This research note is restricted to the personal use of .

Table 3: Comparing Service Mesh, Ingress Gateway/Lightweight API Gateway and Enterprise API Gateway

(Enlarged table in Appendix)

	Service mesh	Ingress gateway/lightweight API gateway	Enterprise API gateway
Topology	▪ Manages east-west traffic ▪ Can manage north-south traffic (via ingress gateway)	▪ Manages north-south traffic ▪ Ingress gateways hand off to east-west traffic when using a service mesh ▪ Lightweight gateways may hand off but are also used to manage east-west traffic	▪ Manages north-south traffic ▪ May be used for internal east-west traffic
Monitoring	▪ Monitors service-to-service traffic ▪ Automatically instruments traffic ▪ Supports distributed tracing	▪ Monitors two-way traffic through the gateway ▪ Can support distributed tracing	▪ Monitors two-way traffic through the gateway ▪ Supports tracing
Security	▪ Tokens and credential verification ▪ Encryption ▪ Key and certificate management	▪ Authentication ▪ Token verification ▪ Some API protection	▪ Authentication ▪ Authorization ▪ Encryption ▪ Token verification ▪ API attack protection
Traffic management	▪ Dynamic discovery ▪ Routing ▪ Load balancing ▪ Retries ▪ Circuit breaker	▪ Basic routing ▪ Load balancing ▪ Throttling ▪ Circuit breaker	▪ Advanced routing ▪ Load balancing ▪ Throttling ▪ Circuit breaker

Source: Gartner

See How a Service Mesh Fits Into Your API Mediation Strategy and Decision Point for Selecting the Right API Mediation Technology for more details.

Alternatives to Service Mesh

Before deciding on using a service mesh, you should consider these alternatives:

- Use Kubernetes and other container orchestration technologies.
- Adopt a distributed application platform such as Dapr.
- Use a network-centric mesh — that is, a CNI extension.

This research note is restricted to the personal use of .

Use Kubernetes and Other Container Orchestration Technologies (You Really Don't Need a Service Mesh)

Before committing to a service mesh architecture, it is important to understand what problems you're trying to solve using a service mesh. Weigh the benefits can you realize against the potential overhead and costs associated with implementing, operating and maintaining a service mesh architecture.

A service mesh will not bring any additional benefits where:

- **Applications have limited east-west traffic:** All the communication in your application remains within a single boundary and there is no communication across network boundaries. Transparent encryption and observability can be handled by other means, such as an advanced CNI like Calico or Cilium, and east-west traffic routing can be handled by classical load balancing.
- **Applications have a fixed topology:** The network topology of your application is fixed and subject to very limited change.
- **Applications require minimal code changes:** Applications that are in a sustaining mode or have minimal updates. Few changes/releases indicates less value from traffic routing policies. The exception is where you have a containerized application with minimal security. In this case, a service mesh can be used to introduce secure communications between services.

Even if your application cannot gain sufficient benefits from a service mesh today, you can still rely on the built-in ingress/gateway controller, DNS, service discovery, monitoring and routing features of the container orchestration platform (such as Kubernetes). As you add more services to your application or as new requirements arise, you should reevaluate the suitability of a service mesh.

Adopt a Distributed Application Platform

Service meshes aim to abstract complex service networking concerns from applications or services and delegate that responsibility to the infrastructure in the form of a common data plane implementation (such as a sidecar). Application developers can focus on business logic while platform engineering, NetworkOps, and SecOps deal with how to enable secure and transparent communication among services using a service mesh.

This research note is restricted to the personal use of .

However, not all distributed application challenges can be easily framed as a networking problem and handed off to the infrastructure. Many distributed application capabilities, such as state management, publishers and subscribers (pub/sub), resource binding, and secrets management, are better implemented in the application layer (above layer 7). In this layer, the developers have more direct and flexible access to these capabilities and can use them as building blocks. In these situations, developers should consider adopting a distributed application platform such as Dapr (Distributed Application Runtime) — a Cloud Native Computing Foundation (CNCF) project originally created by Microsoft.

Dapr shares many architectural and functional similarities with service meshes. For example, they both use a sidecar architecture and support mTLS, metrics, distributed tracing, and resiliency for interservice communication. In terms of where they differ, service meshes offer traffic management capabilities such as routing and splitting, where Dapr has modular distributed application building blocks, including:

- Service-to-service invocation
- Workflows
- State management
- Pub/sub
- Secrets management
- Resource binding and triggers
- Configuration stores
- Distributed lock
- Actors
- Cryptography

For more information, see the Dapr webpage for Building Blocks.

Dapr is a good alternative to service mesh if you want to take advantage of these distributed application building blocks while still being able to choose your preferred language and platform for your application. You can use Dapr with or without a service mesh.

This research note is restricted to the personal use of .

Use a Network-Centric Mesh — CNI Extension

Modern CNIs, such as Antrea, Calico and Cilium, can meet some organizations' service mesh requirements when they are light on service routing and high on network security. Those CNIs implement Kubernetes network policies and can encrypt traffic between pods in a transparent manner. Organizations wanting to encrypt their traffic between pods and secure traffic between services in Kubernetes often don't need all the features offered by service mesh. They should look at implementing an advanced CNI. eBPF-based CNI doesn't have the management or performance overhead of a full feature, proxy-based, service mesh.

Strengths

The strengths of service mesh relate back to the best practices that a service mesh implements, and the automation and standardization of these best practices. The strengths of service mesh can be summarized as follows:

- Provide reasonable and rational defaults and best practices for managing east-west communications between services.
- Standardize and abstract away cross-cutting concerns, including logging, connectivity, observability and security policies.
- Decouple service naming and routing from application logic, allowing application development to concentrate on functionality.
- Implement consistent security policies among services via the use of standard encryption techniques and mTLS — including certificate issuance, enabling consistent service-to-service access and communication.
- Enable services to dynamically adapt to failures and changing topologies without the need for operator intervention.

Weaknesses

Service mesh is not a panacea. It is not suitable for many existing applications and will complicate, rather than simplify, architectures when used incorrectly. The best practices that service meshes implement are well-known and well-understood, but they still require a degree of expertise to implement and use correctly. The weaknesses of service mesh can be summarized as follows:

This research note is restricted to the personal use of .

- Sidecar proxy patterns are not suitable for use with many applications already in production, such as applications deployed in application servers or applications constructed of multiple processes using interprocess communication and shared memory to communicate. Sidecar proxies also introduce latency and add operational overhead, especially for non-Kubernetes ecosystems.
- Service mesh implementations are gaining maturity; however, they are still difficult to use and operate, and the landscape is evolving rapidly.
- Service meshes remain difficult to manage because many still lack the proper role-centric separation of concern necessary to facilitate collaboration among multiple roles and personas from application DevOps, platform engineering, and NetSec/NetOps teams. The emerging Gateway API for Mesh Management and Administration (GAMMA) specification promises to address these shortcomings.

Guidance

The service mesh pattern is not a new concept. However, the scale and complexity of any significant cloud-native architecture requires new approaches to implement it.

Gartner anticipates that many of the features of the current implementations for service mesh will become standard features of all container platforms. Gartner further anticipates that current implementations will be significantly altered in the coming years. At its core, service mesh is an implementation and standardization of well-known best practices and principles.

Gartner's guidance is as follows:

- **Ensure you are prepared before implementing service mesh:** Successful implementation of a service mesh involves a number of minimum prerequisites. You should be already supporting automated deployment of your services using CI/CD tools. Your platform team should be ready to manage container orchestration platforms such as Kubernetes, preferably with namespaces and multicluster support.
- **Implement service mesh as soon as possible for any significant microservices deployments:** If you already have a significant microservices implementation in place without using a service mesh, add one as soon as possible. For Kubernetes-based implementations, evaluate vendors such as Istio, HashiCorp Consul and Linkerd.

This research note is restricted to the personal use of .

- **Start with the sidecar proxy pattern:** When possible, favor the sidecar proxy pattern over the use of development frameworks or explicit API calls from within service code. This pattern allows your services to evolve independent of the cross-cutting concerns that service mesh solves for in the midterm. As service mesh technologies evolve and become a standard component of container platforms, adopting a sidecar proxy pattern will insulate your developers from changes in how service mesh is implemented.

- **Make use of the full potential of service mesh:** Organizations frequently start looking at service mesh to solve observability problems. However, service mesh patterns also allow organizations to implement policies such as securing communications between services with mTLS and using intelligent traffic policies in testing and release scenarios. These capabilities had been beyond the reach of many organizations due to the operational complexity of implementing them on a consistent basis rather than for any technological reason. Service mesh implementations automate these complex best practices, which should be put into place as soon as possible.

- **Develop cohesive services:** Apply the principles of domain-driven design to your architecture, and develop cohesive services. Focusing on good architectural practices will deliver significantly more value in your environment than any service mesh implementation.

- **Use a platform engineering approach:** Manage your service mesh as a component of the platform that will be used by multiple agile software development teams. Treat the service mesh as a core feature of your PaaS approach. See Adopt Platform Engineering to Improve the Developer Experience for details.

Evidence

[1] Service Mesh Data Plane vs. Control Plane, Envoy Proxy.

[2] Handling Ingress Traffic, Linkerd Documentation.

[3] Understanding the Eight Fallacies of Distributed Systems, Simple Oriented Architecture.

[4] Why Mechanical Sympathy? Mechanical Sympathy.

Document Revision History

When to Use a Service Mesh in Cloud-Native Architectures - 27 July 2022

This research note is restricted to the personal use of .

Recommended by the Authors

Some documents may not be available as part of your current Gartner subscription.

Designing Services and Microservices to Maximize Agility

MASA: Create Agile Application Architecture With Composable Apps, APIs and Services

Solution Path for Increasing Application Agility Using Microservices Principles

Using Emerging Service Connectivity Technology to Optimize Microservice Application Networking

Decision Point for Selecting the Right API Mediation Technology

How a Service Mesh Fits Into Your API Mediation Strategy

Solution Path for Agile Web Application Architecture

Emerging Technologies: Service Mesh Is Being Pushed by Containers and Key Open-Source Projects

© 2025 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "Guiding Principles on Independence and Objectivity." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .

Table 1: Service Mesh and the Fallacies of Distributed Computing

Fallacies	Service mesh functionality
■ The network is reliable. ■ Latency is zero. ■ Bandwidth is infinite. ■ Transport cost is zero.	■ Configurable resiliency patterns to deal with failures using retries, load balancing and circuit breakers. ■ Additional error-handling patterns to deal with network outages and variations in network reliability and capacity.
■ The network is secure.	■ Systematically enforced security policies across all services within the service mesh.
■ Topology doesn't change.	■ Dynamically adjusts the network to changing topology and application requirements.
■ There is one administrator.	■ API-driven access to the control plane to allow developers and platform owners and operators to jointly manage the entire structure of the service mesh and dynamically reconfigure as needed.
■ The network is homogeneous.	■ Abstraction of the application from Open Systems Interconnection model Layer 3/Layer 4 (L3/L4) and any specific implementation details.

This research note is restricted to the personal use of .

Source: Gartner

Table 2: Example Defaults for Service Mesh

Setting	Example default set by platform engineering	Customized settings by product teams
Service-to-service authentication	Set mTLS by default for all services to force services to encrypt their traffic by default.	Assign service or app identity. Set tags or labels as part of the CI/CD pipeline to provide metadata and context for policy selection and enforcement.
Ingress/egress rules	Set specific ports that traffic can pass through. For example, only traffic on port 80 or port 443 will be allowed.	Determine which services are exposed via the ingress/egress gateways.
Routing	Root-level Uniform Resource Identifier (URI) prefix delegation to namespaces owned by individual development teams.	Control the routing under the preassigned root URI prefix.
Monitoring	Export default monitoring statistics such as number of calls to a service.	Customize monitoring configuration and access metrics for services they own and operate.
Circuit breaking	Set a circuit breaker to trip when the maximum number of pending connections goes over a specific threshold.	Fine-tune the threshold for their services.

Source: Gartner

This research note is restricted to the personal use of .

Table 3: Comparing Service Mesh, Ingress Gateway/Lightweight API Gateway and Enterprise API Gateway

	Service mesh	Ingress gateway/lightweight API gateway	Enterprise API gateway
Topology	■ Manages east-west traffic ■ Can manage north-south traffic (via ingress gateway)	■ Manages north-south traffic ■ Ingress gateways hand off to east-west traffic when using a service mesh ■ Lightweight gateways may hand off but are also used to manage east-west traffic	■ Manages north-south traffic ■ May be used for internal east-west traffic
Monitoring	■ Monitors service-to-service traffic ■ Automatically instruments traffic ■ Supports distributed tracing	■ Monitors two-way traffic through the gateway ■ Can support distributed tracing	■ Monitors two-way traffic through the gateway ■ Supports tracing
Security	■ Tokens and credential verification ■ Encryption ■ Key and certificate management	■ Authentication ■ Token verification ■ Some API protection	■ Authentication ■ Authorization ■ Encryption ■ Token verification ■ API attack protection

This research note is restricted to the personal use of .

Traffic management	■ Dynamic discovery ■ Routing ■ Load balancing ■ Retries ■ Circuit breaker	■ Basic routing ■ Load balancing ■ Throttling ■ Circuit breaker	■ Advanced routing ■ Load balancing ■ Throttling ■ Circuit breaker

Source: Gartner

This research note is restricted to the personal use of .